

Asymptotic Notations

1. Big-O Asymptotic notation is used to describe the performance (time or space) of an algorithm as input size

'n' becomes very large

→ It tells us how a function behaves as $n \rightarrow \infty$

Types

1. Big-O Notation :

Upper Bound (Worst case)

→ It shows that Maximum time an algorithm may take.

Definition:

The function $f(n) = O(g(n))$ if there exist two positive constants $c > 0$ & n_0 with the constraint such that:

$$f(n) \leq c \cdot g(n) \quad \text{for all } n \geq n_0$$

Note:

Here $f(n)$ = actual function

$g(n)$ = growth rate function

$g(n)$ is just the simplified growth term we compare with usually with.

→ 1, $\log n$, n , n^2 , $n \log n$, 2^n , $n!$

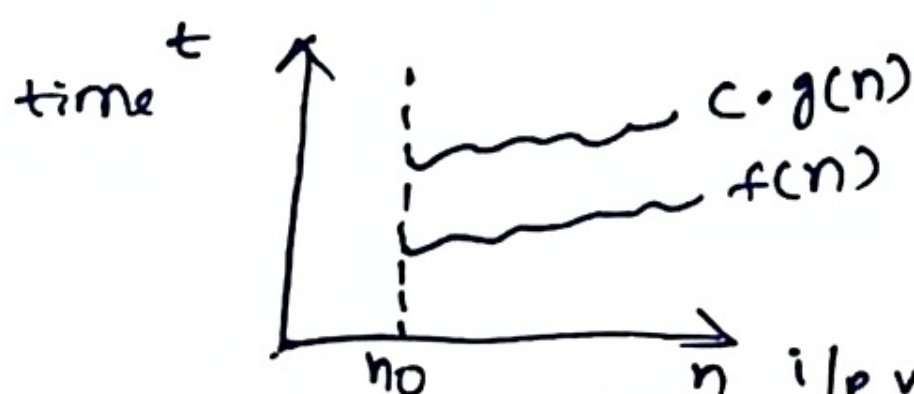


fig: Big-O Notation

eg: Given $f(n) = 3n + 2$ then p.T $f(n) = O(n)$

Sol: Step1: Identifying all terms

The function has

$$\rightarrow 3n$$

$$\rightarrow 2$$

Step2 find the fastest growing term, choose $g(n)$

$g(n)$ = growing highest term in function

$$g(n) = n.$$

Step3: My function formula

$$f(n) \leq c \cdot g(n)$$

$$3n + 2 \leq c \cdot n \rightarrow \textcircled{1}$$

Here $c > 0$, $n \geq n_0 \therefore n \geq 1$

Step4: choose constants c , n value

$$c = 5 \quad 5 > 0(T), \quad n = 1 \quad 1 \geq 0(T)$$

$\therefore$ Apply it in eq $\textcircled{1}$

$$3(1) + 2 \leq 5(1)$$

$$5 \leq 5(T)$$

$$\therefore f(n) = O(g(n))$$

$$\boxed{f(n) = O(n)}$$

2. Big-Omega Notation (Ω)

(3)

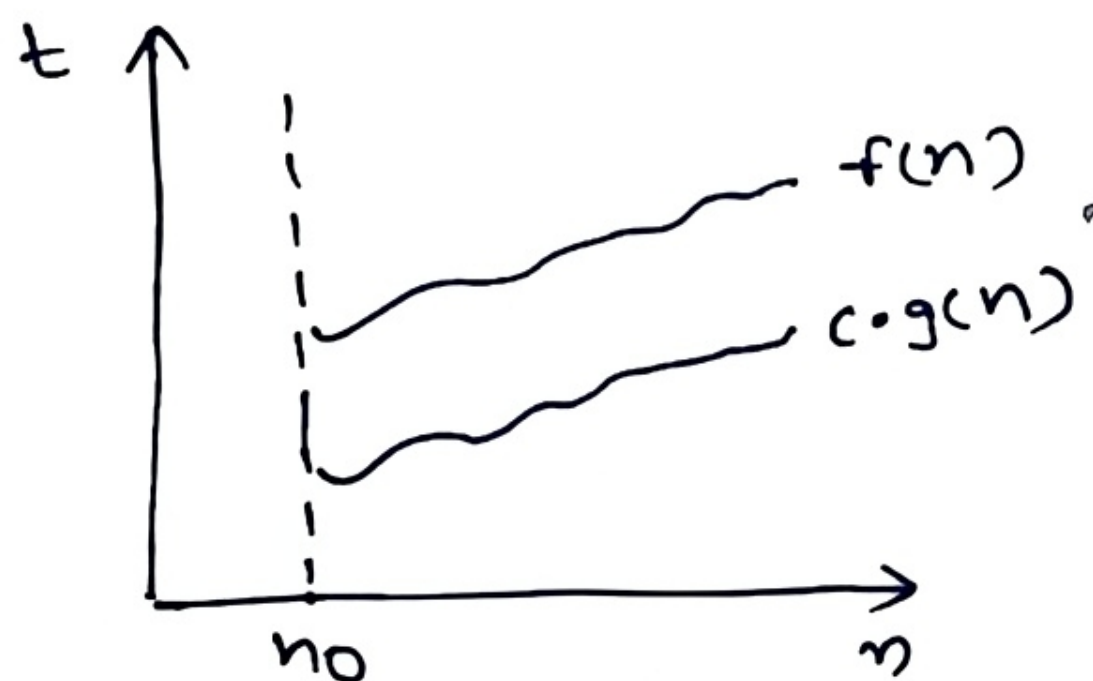
→ Lower bound (Best case) → Minimum time required

This notation is used to find the lower bound behaviour of $f(n)$

Definition

The function $f(n) = \Omega(g(n))$ if there exists two positive integer constant $c > 0$ & n_0 with the constraint such that

$$f(n) \geq c \cdot g(n) \quad \forall n \geq n_0$$



eg: Given $f(n) = 3n^2 + 5n + 7$ then p.T $f(n) = \Omega(n^2)$

Sol: $f(n) = 3n^2 + 5n + 7$

Choose $g(n) = n^2$ (Highest power term is n^2 in the function)

We have to prove: $f(n) \geq c \cdot g(n)$

$$3n^2 + 5n + 7 \geq c \cdot n^2$$

for some c & n_0 $\therefore c > 0$, $n \geq n_0$
 $n \geq 1$

$$3n^2 \geq c \cdot n^2$$

$$\text{Here } c = 3 \quad n = 1$$

$$3n^2 \geq 3n^2$$

$3 \geq 3$ (T) it satisfies the function.

$$\therefore f(n) \geq c g(n) \quad \forall n \geq n_0$$

$$\therefore \boxed{f(n) = \Omega(n^2)}$$

(3) Big-Theta Notation (Θ)

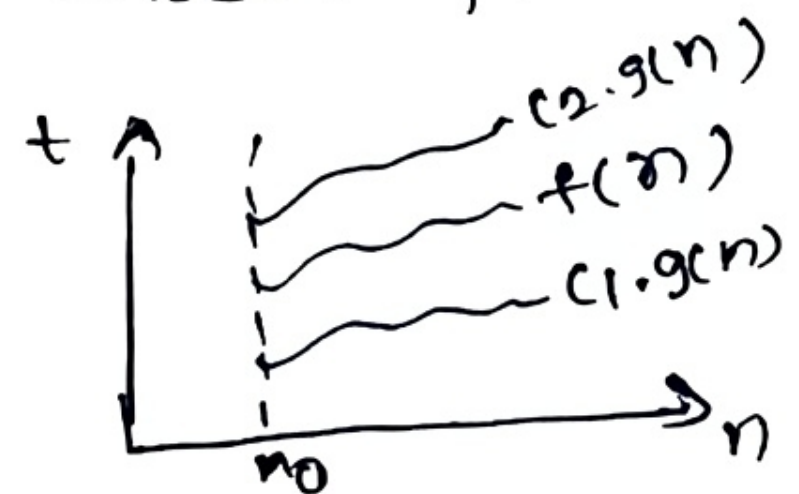
→ Tight Bound (or) exact growth,

→ Average case running time of algorithms

→ It gives tight bound - both upper bound

or lower bound

Definition:



The function

$f(n) = \Theta(g(n))$ if there exist ^{three} positive

Constants c_1, c_2 or n_0 such that

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \quad \text{for all } n \geq n_0.$$

eg: Given $f(n) = 3n^2 + 5n + 7$ then P.T $f(n) = \Theta(n^2)$

Sol: choose $g(n) =$ Higher power term of $f(n)$

$$\boxed{g(n) = n^2}$$

$$\therefore c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$$

1st evaluate $f(n) \leq c_2 \cdot g(n)$ upper bound.

2nd evaluate $f(n) \geq c_1 \cdot g(n)$ lower bound

upper bound:

$$f(n) \leq c_2 \cdot g(n)$$

$$3n^2 + 5n + 7 \leq c_2 \cdot n^2$$

$$\text{Here } c_2 = 15, n = 1$$

$$15 \leq 15$$

$$\therefore \boxed{f(n) = O(n^2)} \rightarrow \textcircled{1}$$

Lower bound

$$f(n) \geq c_1 \cdot g(n)$$

$$3n^2 + 5n + 7 \geq c_1 \cdot n^2 \quad c_2 = 15, n = 1$$

$$15 \geq 15$$

$$\boxed{f(n) = \Omega(n^2)} \rightarrow \textcircled{2}$$

combine $\textcircled{1}$ & $\textcircled{2}$ we prove that

$$\boxed{f(n) = \Theta(n^2)}$$

Key points:

Best case	→	Minimum time	→	Big- Ω	$\Omega(1)$	'1' search
Worst case	→	Maximum time	→	Big- O	$\cdot \Theta(n)$	$n/2$ search
Average case	→	Expected time	→	Θ	$\Theta(n)$	$n/2$ search

Compare Big-O, Big-Ω, Big-Θ.

Big-O, Big-Ω & Big-Θ are asymptotic notations used to describe how an algorithm's running time (or, Space) grows as the 'input size' 'n' increases

Feature	Big-O (O)	Big-Ω	Big-Θ
Definition	upper bound of running time	Lower bound of running time	Tight (exact) bound of running time
Represents	Worst-case complexity	Best-case complexity	Exact Asymptotic complexity
Meaning	Algorithm takes at most this much time	Algorithm takes minimum time (at least) $f(n)$	Algorithm takes average amount of time (at least & at most) $f(n)$
Bound type	upper bound	Lower bound	upper & lower bound
Example	Linear Search: $O(n)$	Linear Search: $\Omega(1)$	Array traversal: $\Theta(n)$
Mathematical Condition	$f(n) \leq c \cdot g(n)$ $\forall n \geq n_0$ $c > 0$	$f(n) \geq c \cdot g(n)$ $\forall n \geq n_0$ $c > 0$	$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$

Space complexity:

(7)

* Space complexity is the total amount of memory required by an algorithm to execute, as a function of the i/p size.

* To compute space complexity, we consider two main factors:

1. fixed part / constant
2. variable part.

fixed part / constant:

* The fixed part is the portion of memory that does not depend on the input size (n). It remains constant.

* The fixed part typically consists of:

1. Simple variables
2. Constants
3. Program instructions
4. fixed-size structures.

Variable part:

* The portion of memory that depends on the input size (n).

* Variable consists of:

1. Arrays whose size depends on ' n '.

2. Dynamic memory allocation (malloc, calloc)

Space complexity formula:

$$S(P) = C + Sp$$

Here $S(P)$ = Total space complexity of the program

P = Algorithm name.

C = constant (fixed) space

Sp = variable part

Basic rules for space calculations:

1. Size of variable like n, a, b assigns $\rightarrow$ 1 word
2. Array size $\rightarrow n$ words
3. for loop variable assignment operator assigns $\rightarrow$ 1 word.

Eg: calculate space complexity for sum of n elements in the array.

Algorithm sum(a, n)

{

$S \leftarrow 0$

 for $i \leftarrow 0$ to n do

$S \leftarrow S + a[i]$

 returns;

}

Constant part	Variable part
$S \leftarrow 0$: 1 word	
$i \leftarrow 0$: 1 word	$a[i]$: n words
n : 1 word	

Step 1: $S(P) = C + Sp$

$$S(\text{sum}) = C + Sp$$

$$\therefore S(\text{sum}) = (1+1+1) + n = 3 + n$$

$$S(P) = (n+3) \text{ Words}$$

eg: Sum of n elements. calculate space complexity

Algorithm Sum(a, n)

// Problem description: The algorithm for sum of n elements

// in the array

// Input: An array a & n is total no. of elements in the array

// output: Returning the values to the sum 's' & stored in that variable & display output.

{

$S \leftarrow 0$

for $i \leftarrow 1$ to n do

$S \leftarrow S + a[i]$

return S ;

}

The space requirement for the above algorithm is

$$S(P) = C + Sp$$

$$S(\text{sum}) = C + Sp$$

Count:

1. Constant part:

$\therefore S \leftarrow 0$; Assigning the value — 1 word

$i \leftarrow 1$ — 1 word

' n ' variable $\rightarrow$ 1 word

2. Variable part:

array value $a[i] \rightarrow n$ word (a is depends on i)

$$\therefore S(P) = (1+1+1) + n$$

$$S(\text{sum}) = 3 + n$$

$$S(P) = (n+3) \text{ words.}$$

eg: Sum of n elements. calculate space complexity

Algorithm Sum(a, n)

// Problem description: The algorithm for sum of n elements

// in the array

// Input: An array a & n is total no. of elements in the array

// output: Returning the values to the sum 's' & stored in that variable & display output.

{

$S \leftarrow 0$

for $i \leftarrow 1$ to n do

$S \leftarrow S + a[i]$

returns;

}

The space requirement for the above algorithm is

$$S(P) = C + S_P$$

$$S(\text{sum}) = C + S_P$$

Count:

1. Constant part:

$\therefore S \leftarrow 0$; Assigning the value — 1 word

$i \leftarrow 1$ — 1 word

' n ' variable $\rightarrow$ 1 word

2. Variable part:

array value $a[i] \rightarrow n$ word (a is depends on i)

$$\therefore S(P) = (1+1+1) + n$$

$$S(\text{sum}) = 3 + n$$

$$S(P) = (n+3) \text{ words.}$$

Time complexity

- * Time complexity refers to the amount of time an algorithm takes to complete its execution as a function of the size of the input
- * The time complexity is therefore given in terms of frequency count.
- * frequency count is a count denoting number of times of execution statement.
- * Instead, it measures the number of basic operations (like comparisons, assignments, loops, etc.) performed by the algorithm.
- * Time complexity formula:

$$T(P) = C + T_p$$

$T(P)$ = Time complexity of algorithm name

C = compile time

T_p = Run Time

Time complexity can be analyzed in three ways:

1. Best case:

If an Algorithm is taking minimum amount of time for its execution it is called as best case

2) Average case:

If an Algorithm is taking average amount of time for its execution it is called as Average case.

3) Worst case:

If an algorithm is taking maximum amount of time for its execution it is called as worst case.

Common Time complexities:

1. $O(1)$ - Constant time

2. $O(n)$ - Linear time

3. $O(\log n)$ - Logarithmic Time.

Eg: Binary search

4. $O(n^2)$ - Quadratic Time

Eg: Nested loops, matrix addition

Two approaches to calculate Time complexity:

1. frequency count

2. Asymptotic Notations

Rules for Time complexity:

1. Comments: 0 steps
2. for Assignment Statement: 1 step
3. Condition statement: 1 step
4. loop condition 'n' times (n+1) steps
5. Body of loop: n steps

Eg: Calculate Time complexity for sum of n elements in array.

Sol Here time complexity calculated in terms of frequency count tabular format.

Statement No	Statement	Status of execution (s/e)	frequency count (fc)	Total steps (s/e * fc)
1	Algorithm sum (arr)	0	0	0
2.	{	0	-	0
3.	$S \leftarrow 0$	1	1	1
4.	for {i ← 0 to n do	n+1	(n+1)	(n+1)
5.	$S \leftarrow S + arr[i]$	n	n	n
6.	return S	1	1	1
7.	}	0	-	0

Total Steps for algorithm

$$1 + (n+1) + n + 1 = (2n+3)$$

formula: $T(p) = C + T_p$

$$f(n) = (2n+3) \text{ steps}$$

$\therefore$ Time complexity is:

$$T(p) = O(n)$$

eg 2: calculate $\sum_{i=1}^n$ two matrix for the algorithm

Statement no	Statement	Status of execution S/E	frequency count (fc)	Total steps (S/E * fc)
1	Algorithm Add (a, b, c, m, n)	0	—	0
2	{	0	—	0
3	for i ← 1 to m do	1	(m+1)	m+1
4	for j ← 1 to n do	1	m(n+1)	mn+m
5	c[i][j] ← a[i][j] + b[i][j]	1	mn	mn
6	}	0	0	0

Total steps for algorithm:

$$\begin{aligned} & (m+1) + mn + m + mn \\ &= 2mn + 2m + 1 \\ &= 2m + 2mn + 1 \end{aligned}$$

$$\therefore T(P) = (2m + 2mn + 1) \text{ steps}$$

$$f(n) = 2m + 2mn + 1$$

Time complexity for the algorithm is

$$\boxed{T(P) = O(mn)} \quad (\text{or } O(n^2))$$

CONSTRUCT AVL TREE FOR THE FOLLOWING DATA

①

21, 26, 30, 9, 4, 14, 28, 18, 15, 10, 2, 3, 7

Sol: Step 1: Insert 21

②1 Bf = 0 - 0
= 0

Tree is balanced

Step 2: Insert 26

②1 Bf = 0 - 1 = -1
②6 Bf = 0

Tree is balanced

Step 3: Insert 30

②1 Bf = -2
②6 Bf = -1
③0 Bf = 0

Tree is unbalanced

Apply Rotations

RR case
⇒ (Left rotation)
②6 Bf = 0
②1 Bf = 0
③0 Bf = 0

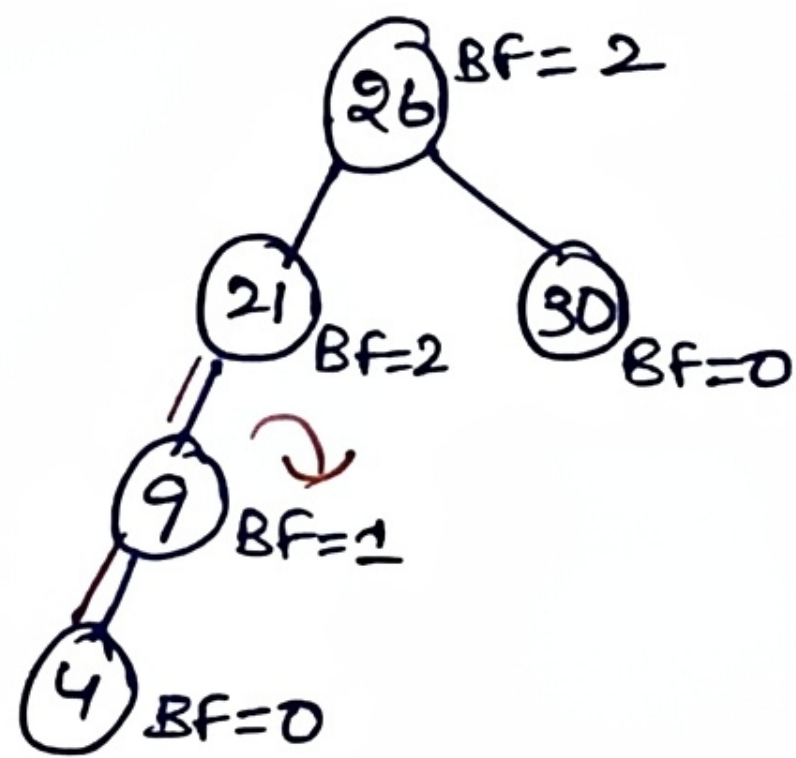
Tree is balanced

Step 4: Insert 9

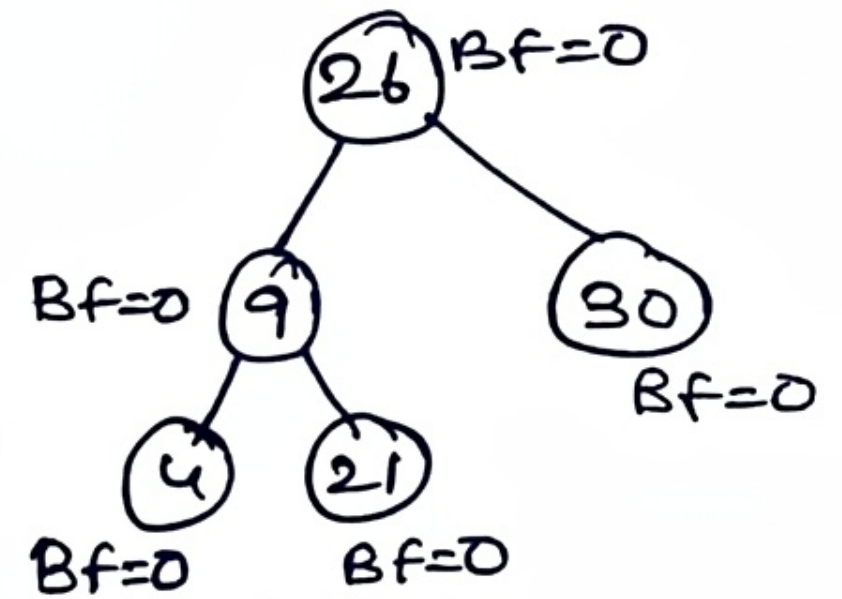
②6 Bf = 2 - 1 = 1
②1 Bf = 1
③0 Bf = 0
⑨ Bf = 0

Tree is balanced

Step 5: Insert 4



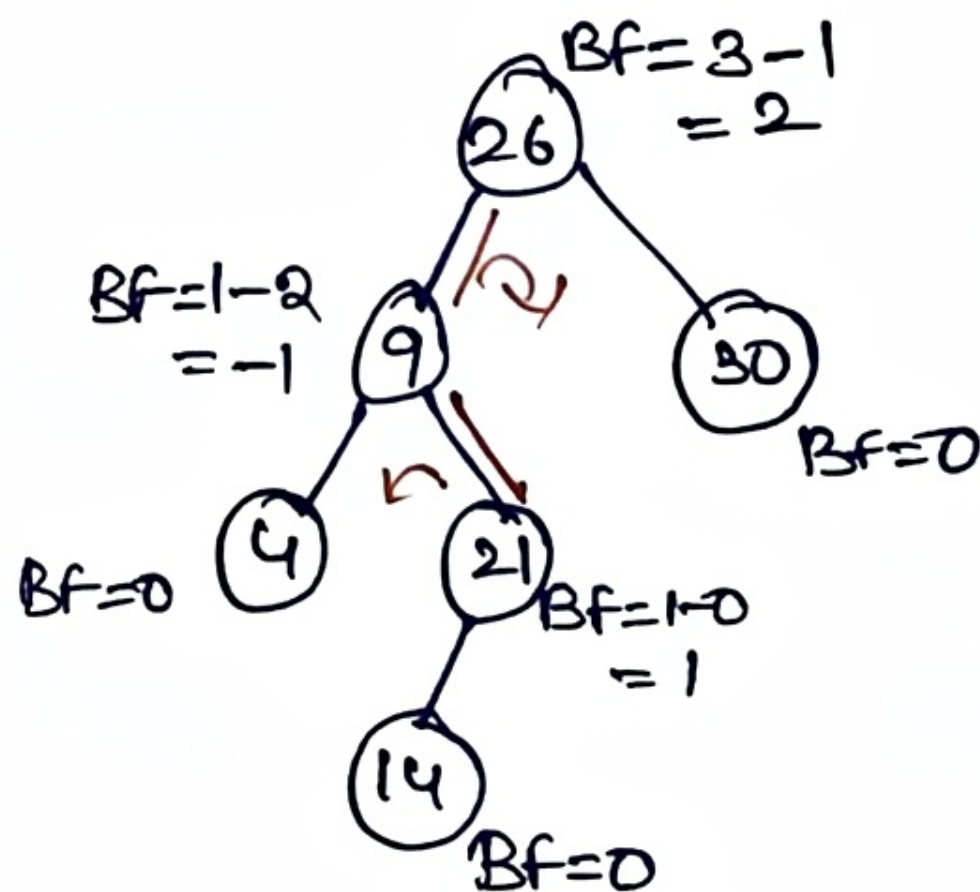
LL case
 $\Rightarrow$
 (Right rotation)



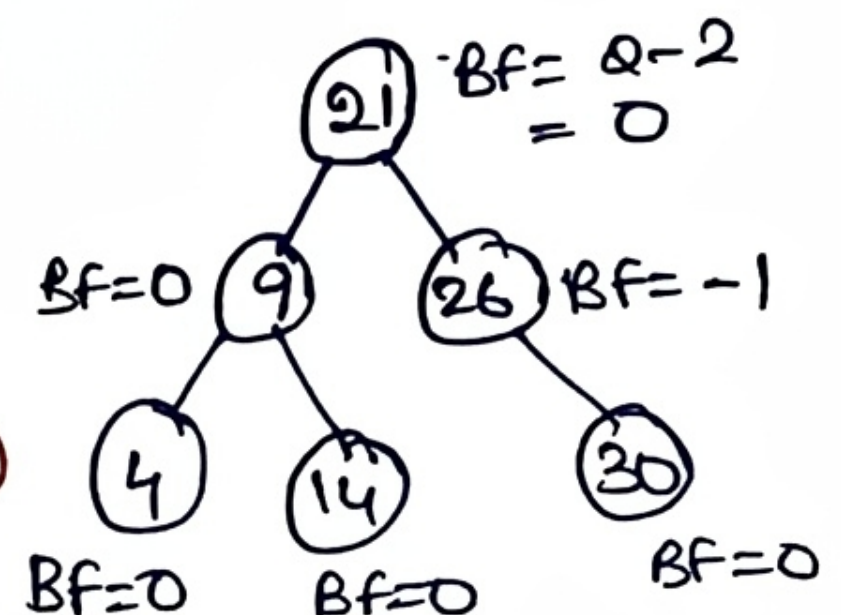
Tree is not balanced at node 21
 Apply rotation at node '21'

Tree is balanced

Step 6: Insert 14



LR case
 $\Rightarrow$
 (RL rotation)

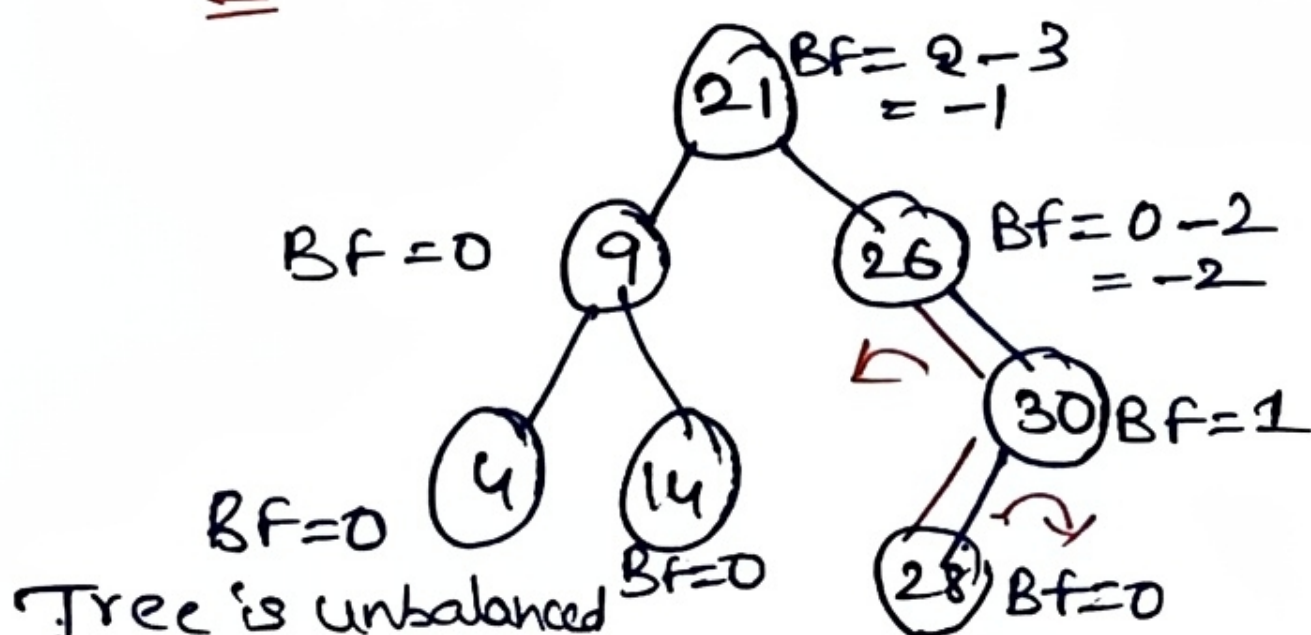


Tree is not balanced at node '26'

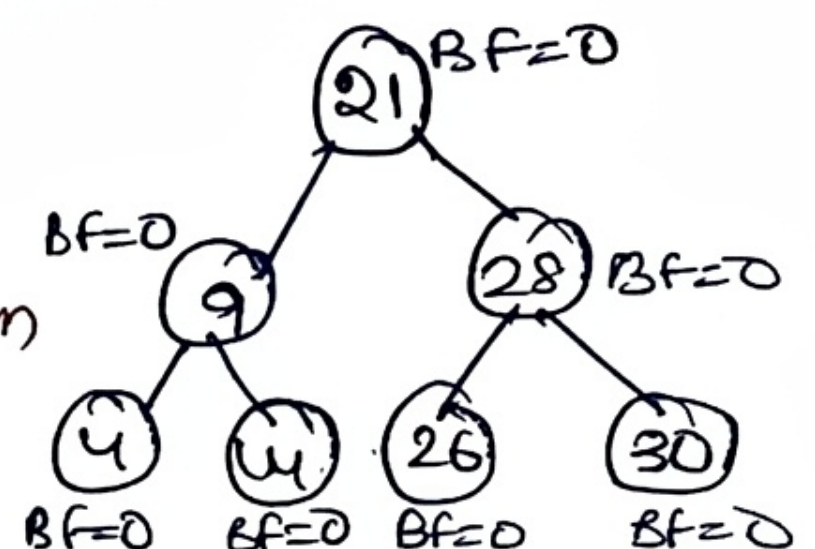
Tree is balanced.

Apply rotation at node '26'

Step 7: Insert 28



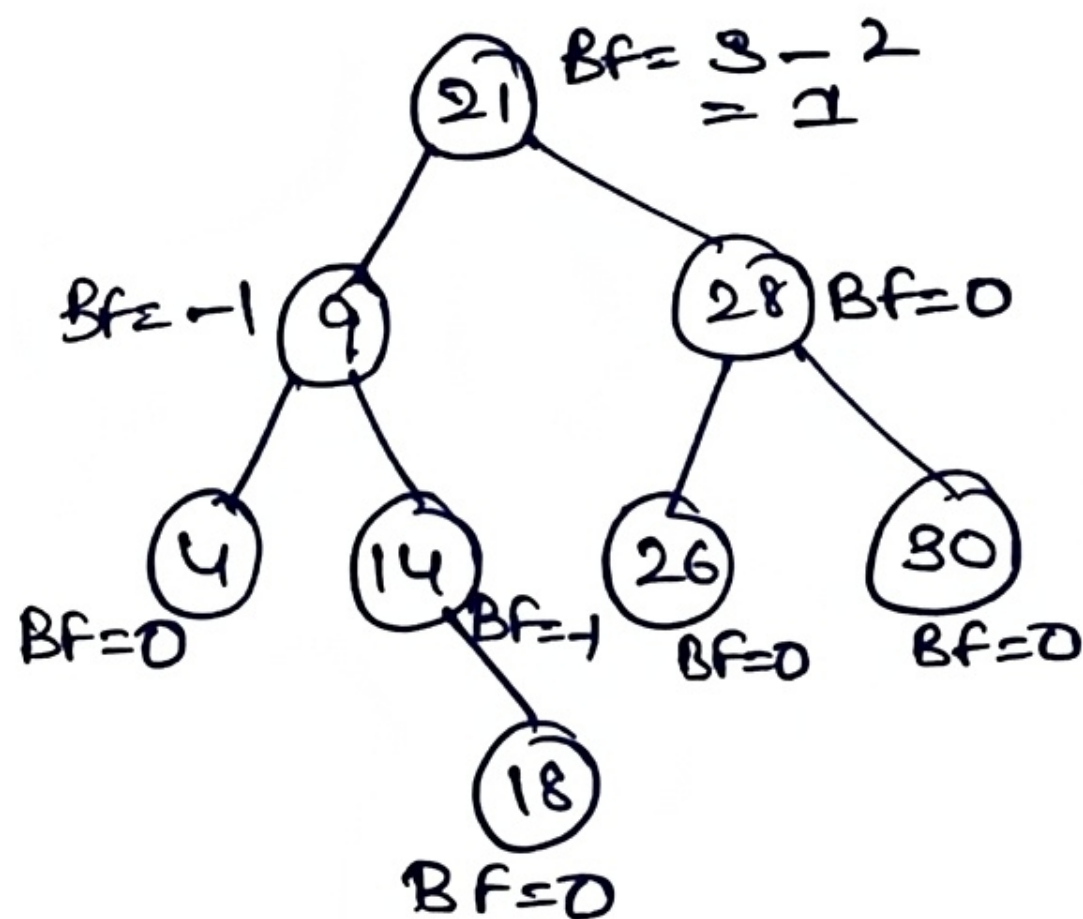
Apply (R) rotation
 $\Rightarrow$
 RL case
 (Left right rotation)



Tree is balanced

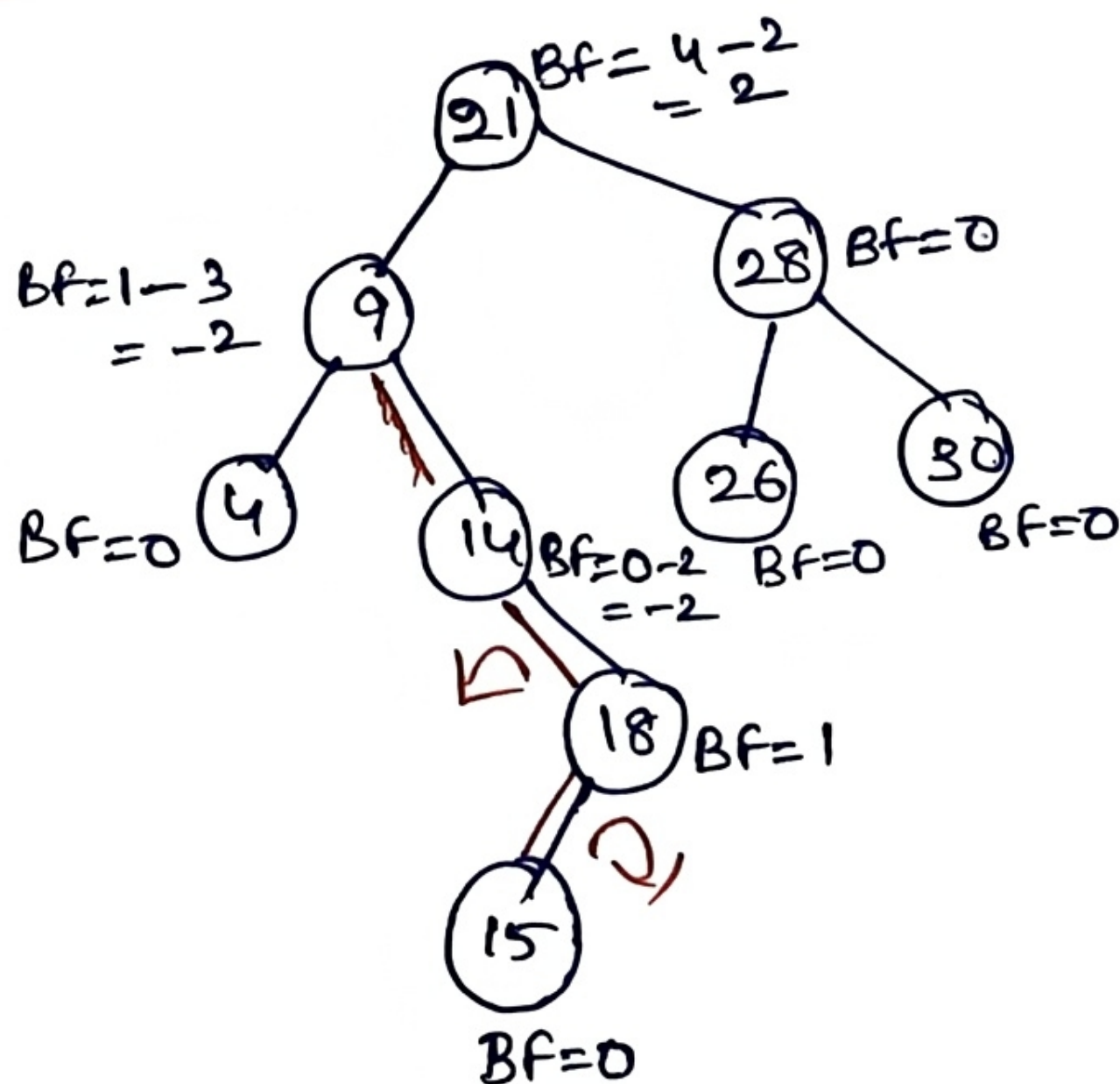
Step 8: Insert 18

(2)



Tree is balanced

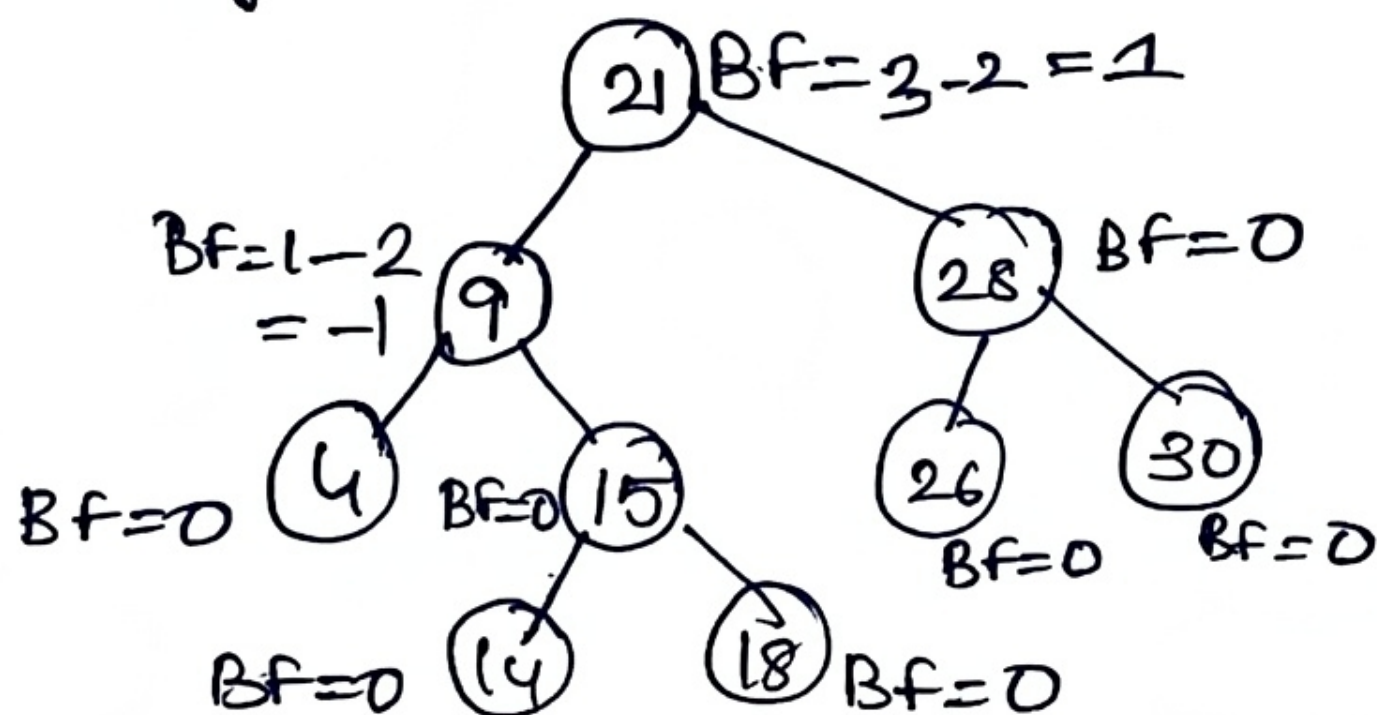
Step 9: Insert 15



Apply LR case
 ⇒ (Right left rotation)

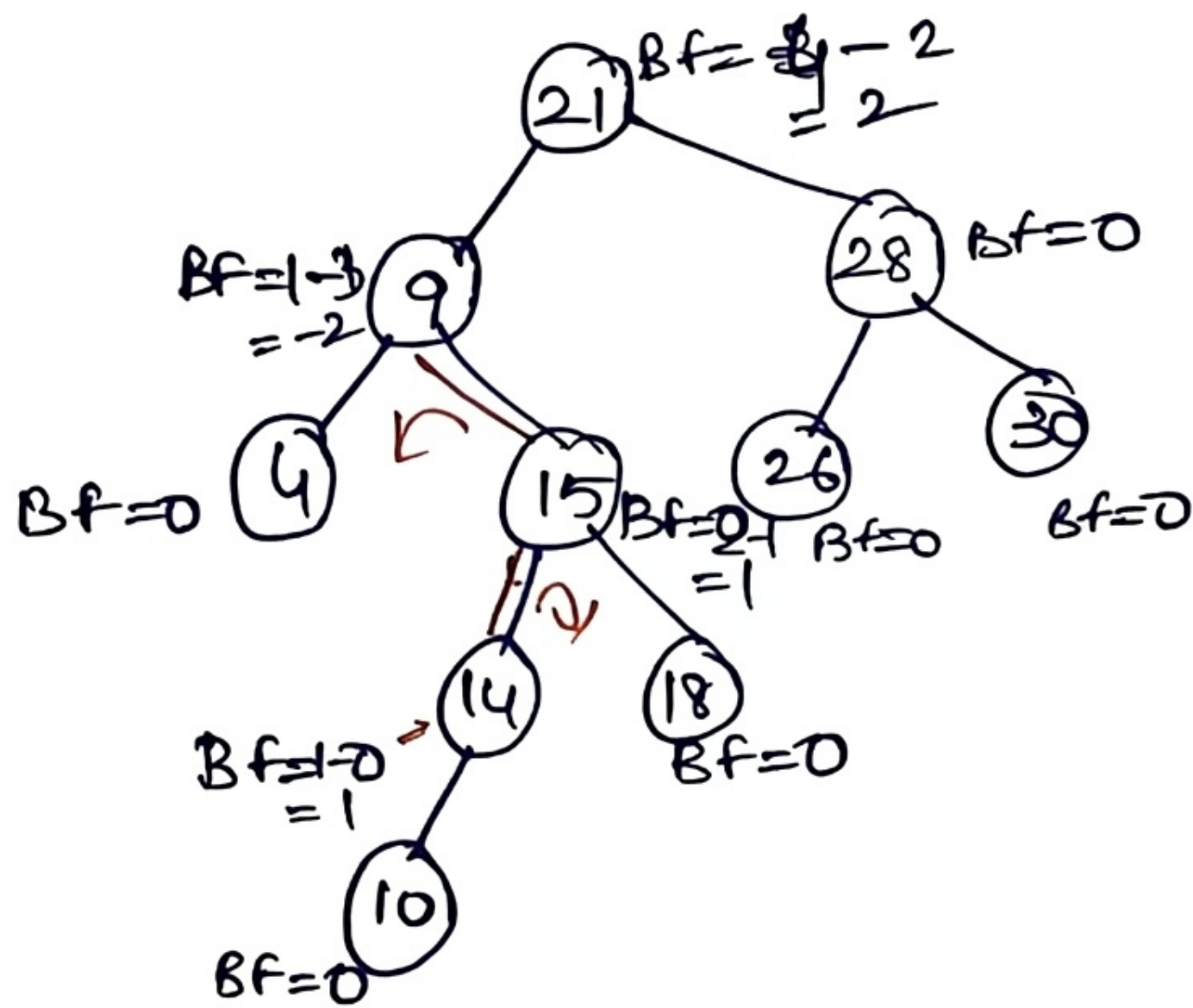
Tree is unbalanced at node 9 is -2

→ Apply rotation for node 9



Tree is balanced

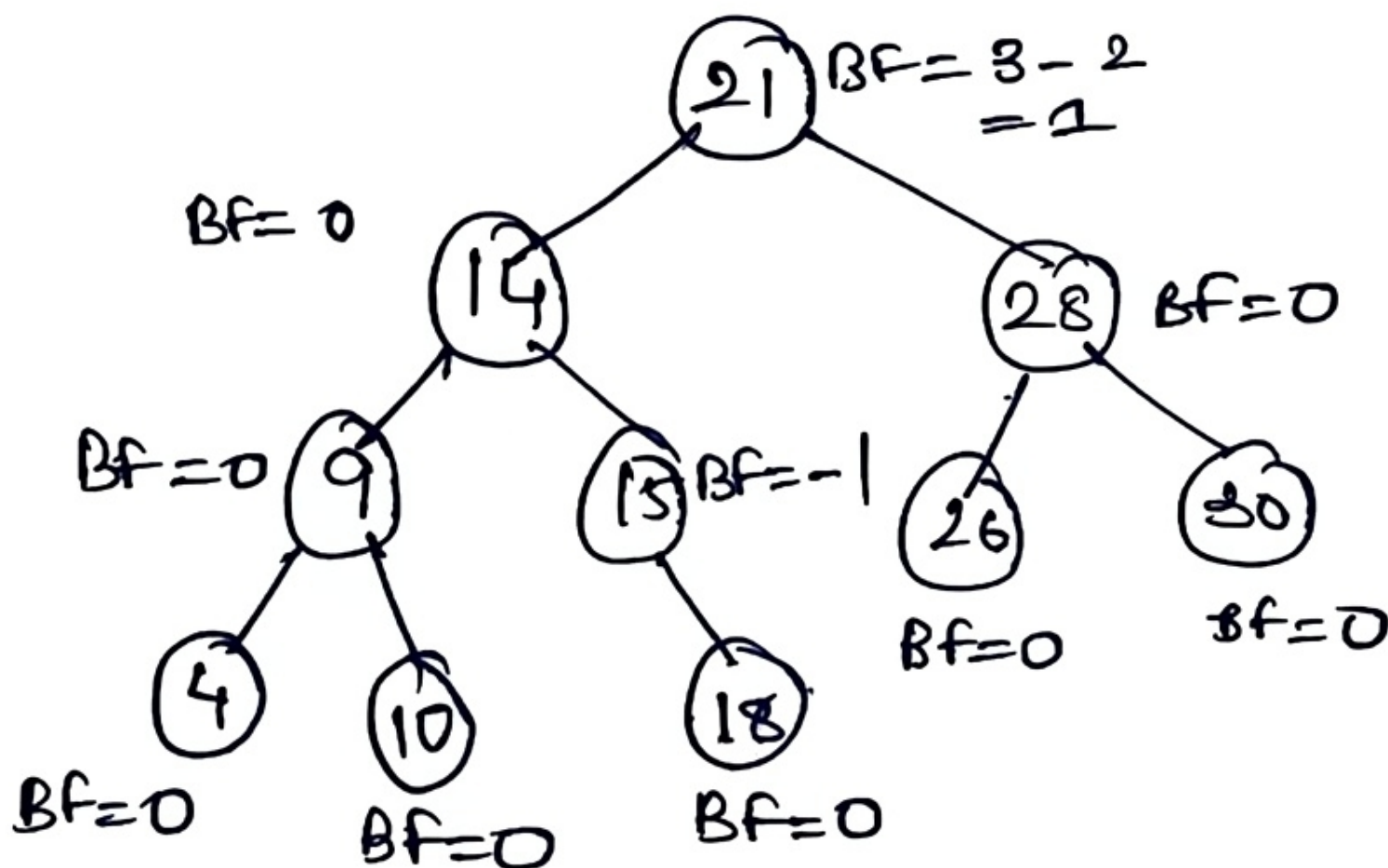
Step 10: Insert 10



Apply
RL
(LR rotation)

Tree is unbalanced at
node '9' is '-2'

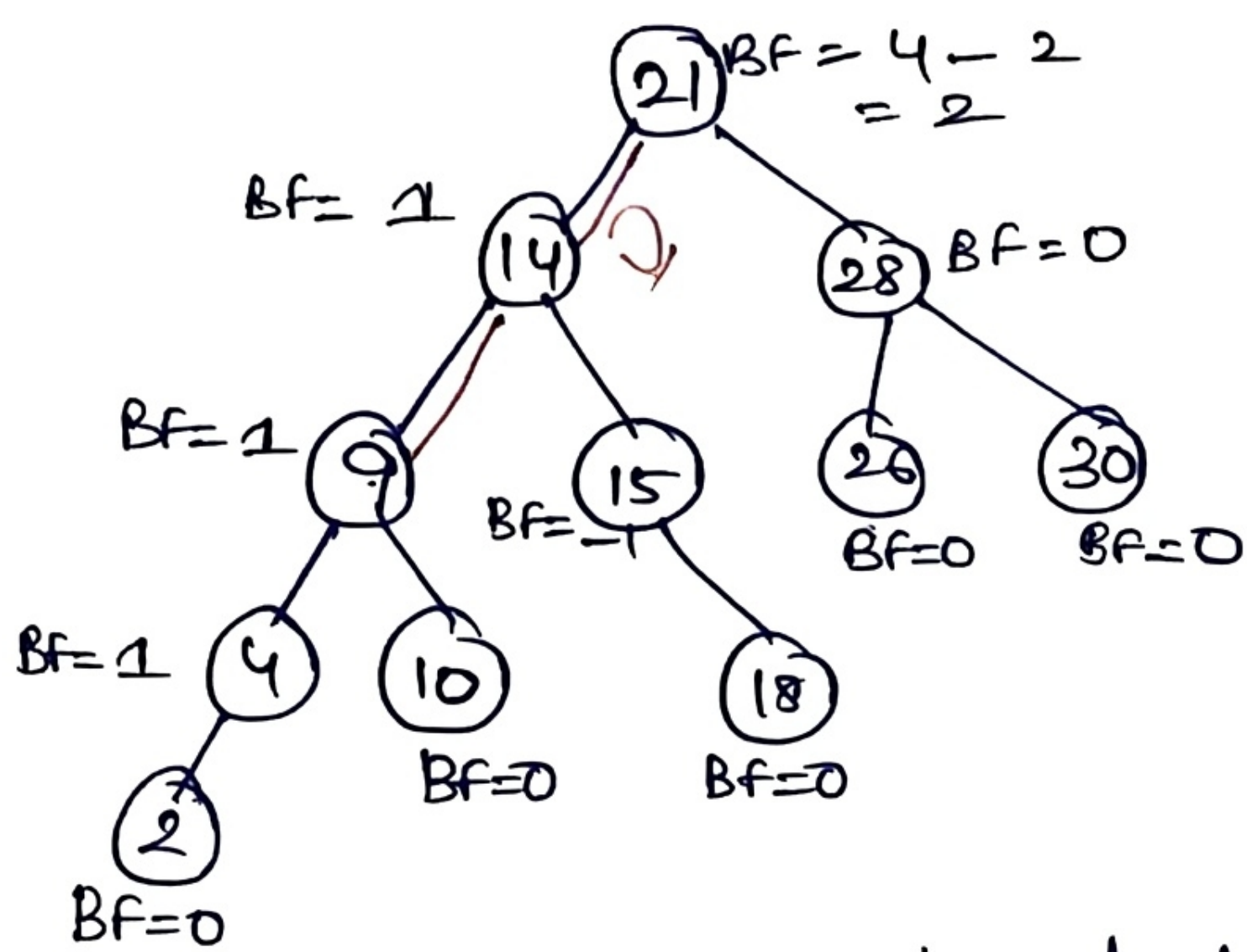
Apply rotations at node '9'



Tree is balanced

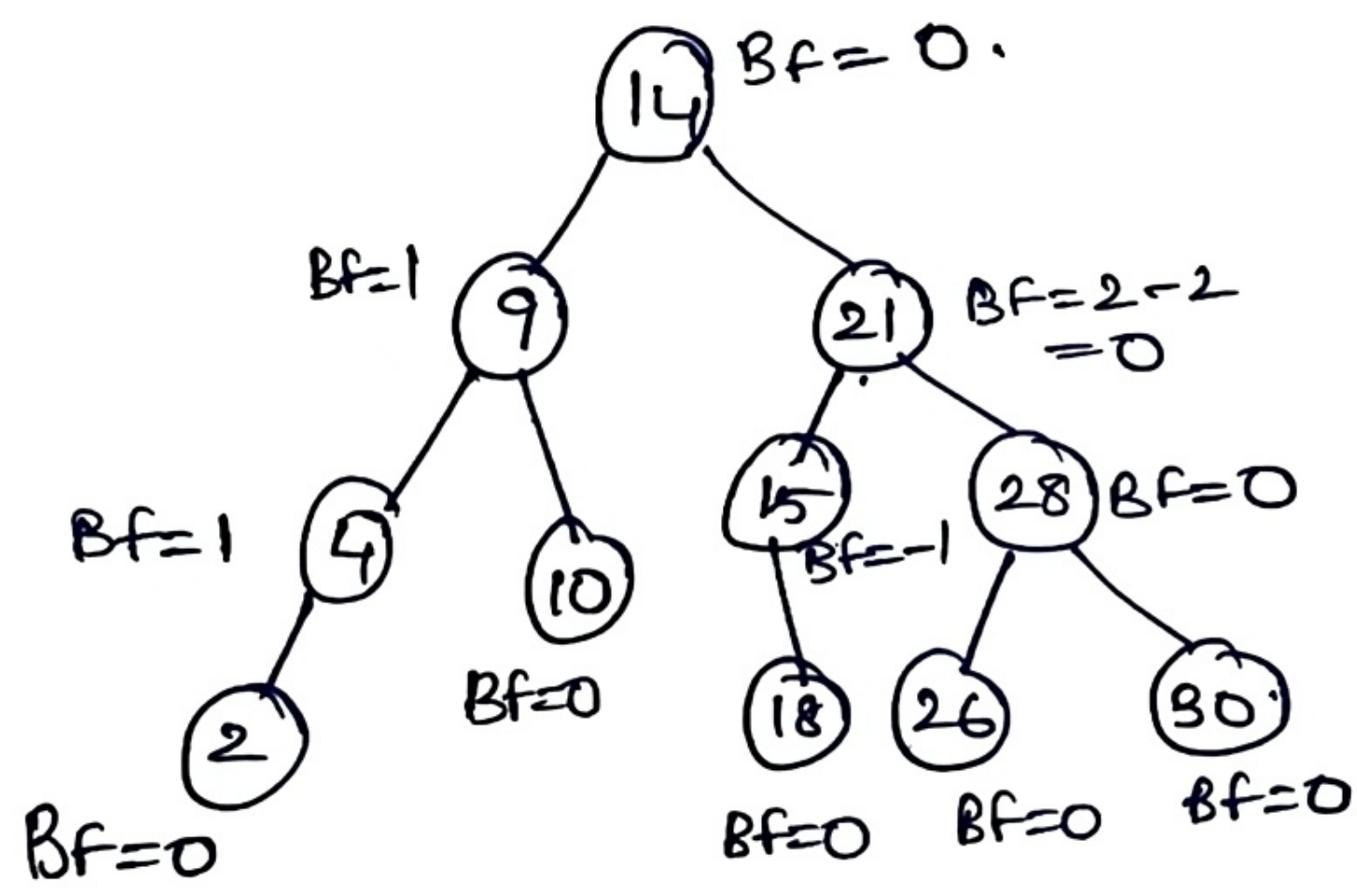
Step 11 Insert 2

8



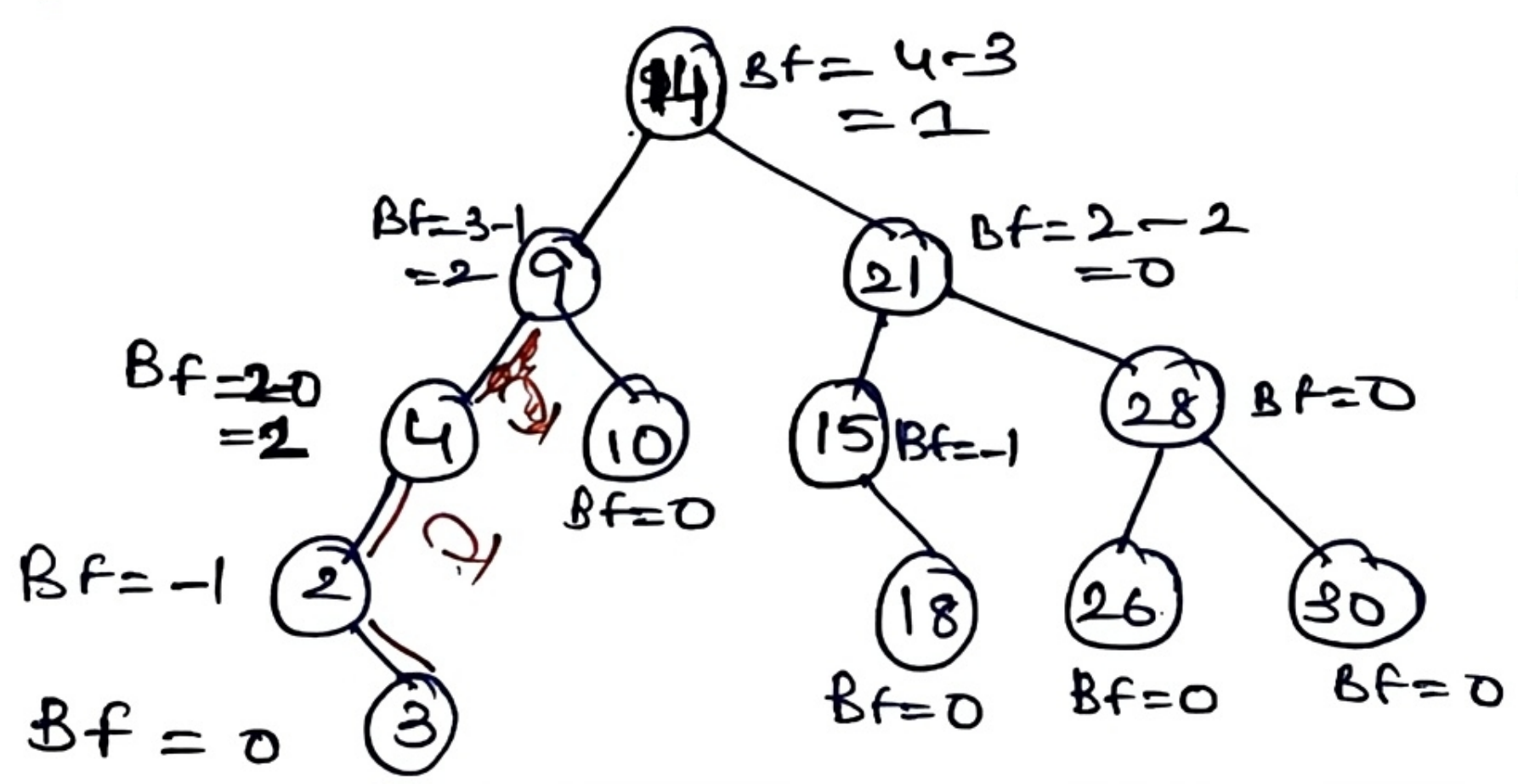
Apply LL case
⇒
(Right rotation)

Tree is unbalanced at Node 21
Apply rotation for node '21'.



Tree is balanced

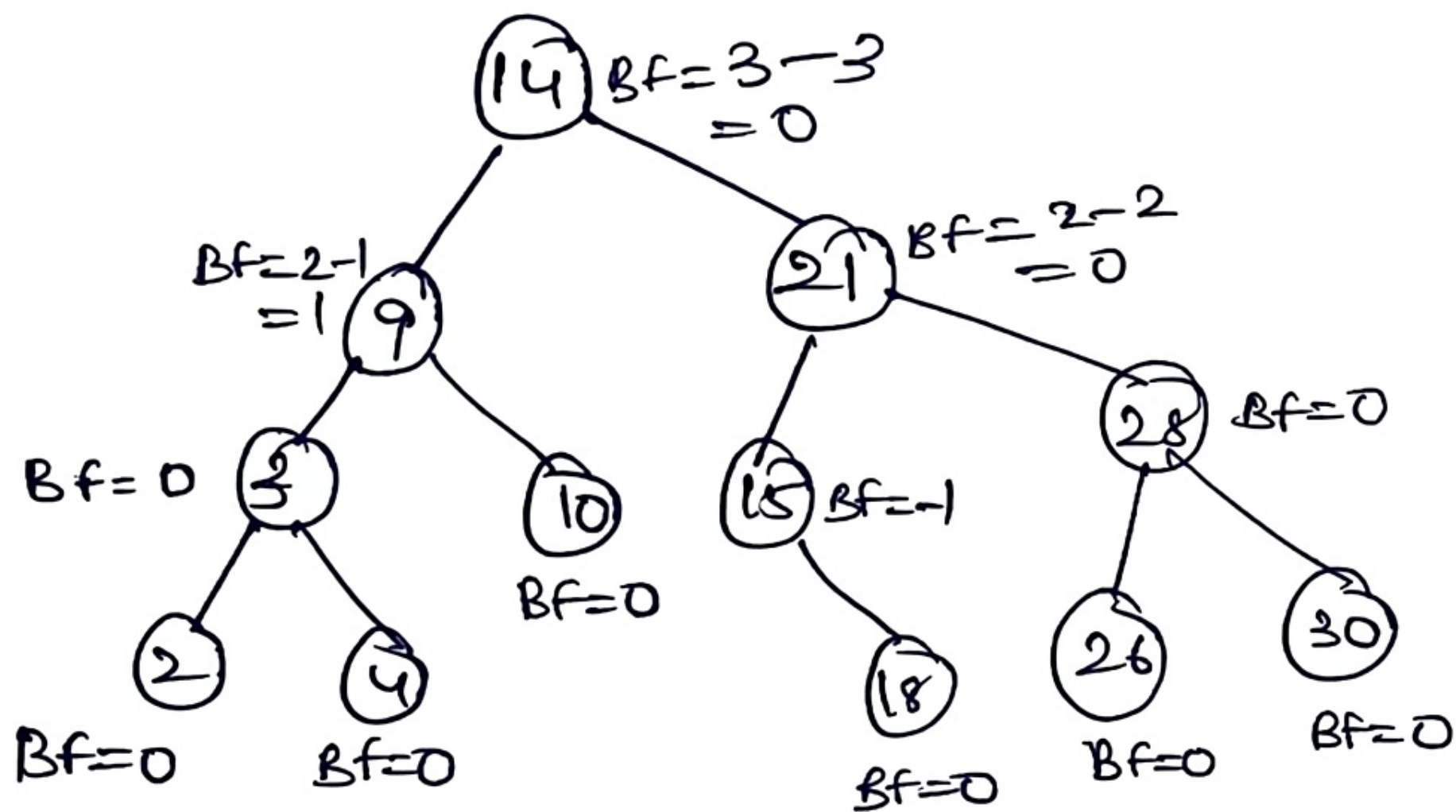
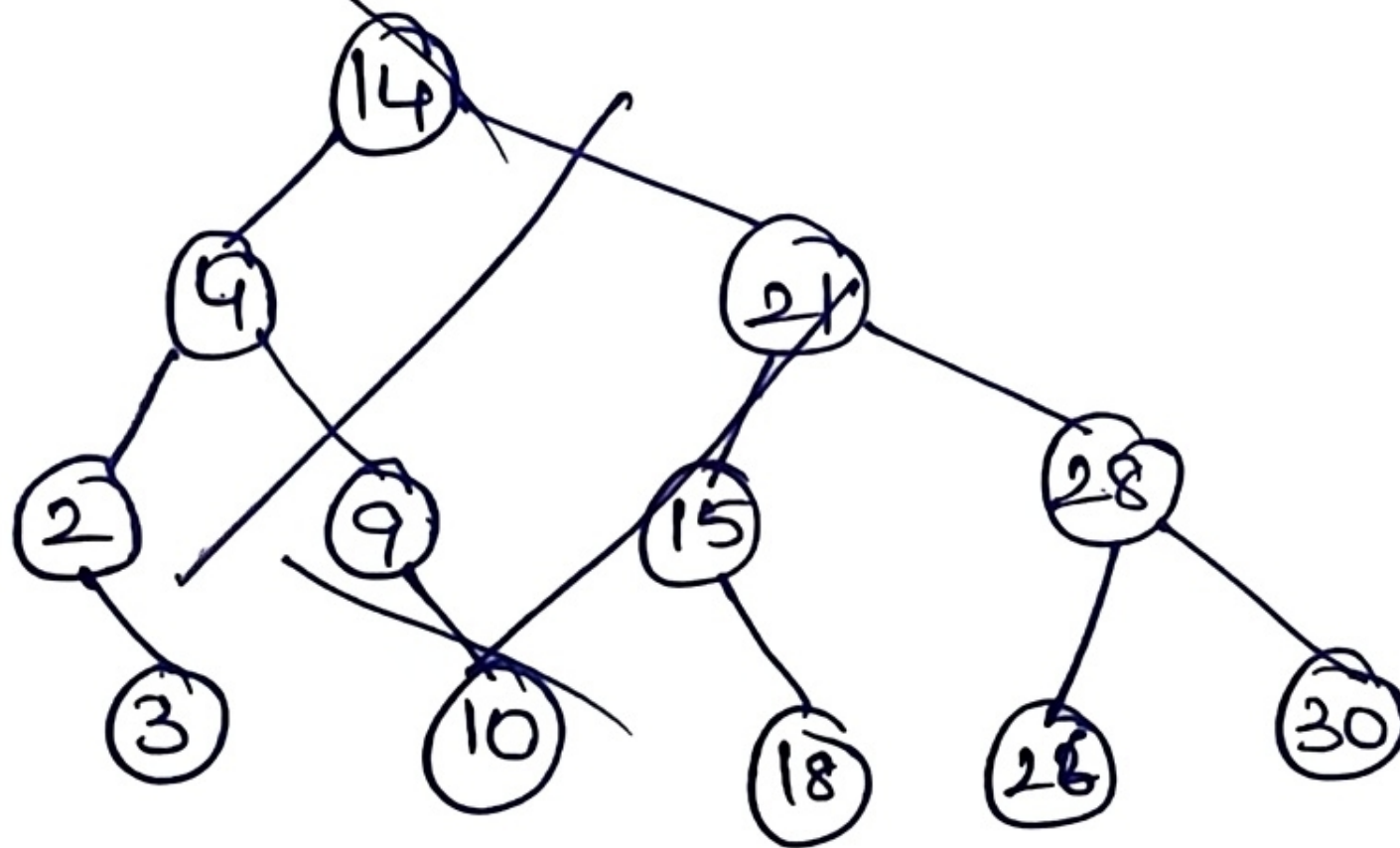
Step 12: Insert 3



Apply LL case
⇒
(Right rotation)

Tree is unbalanced at node '9'

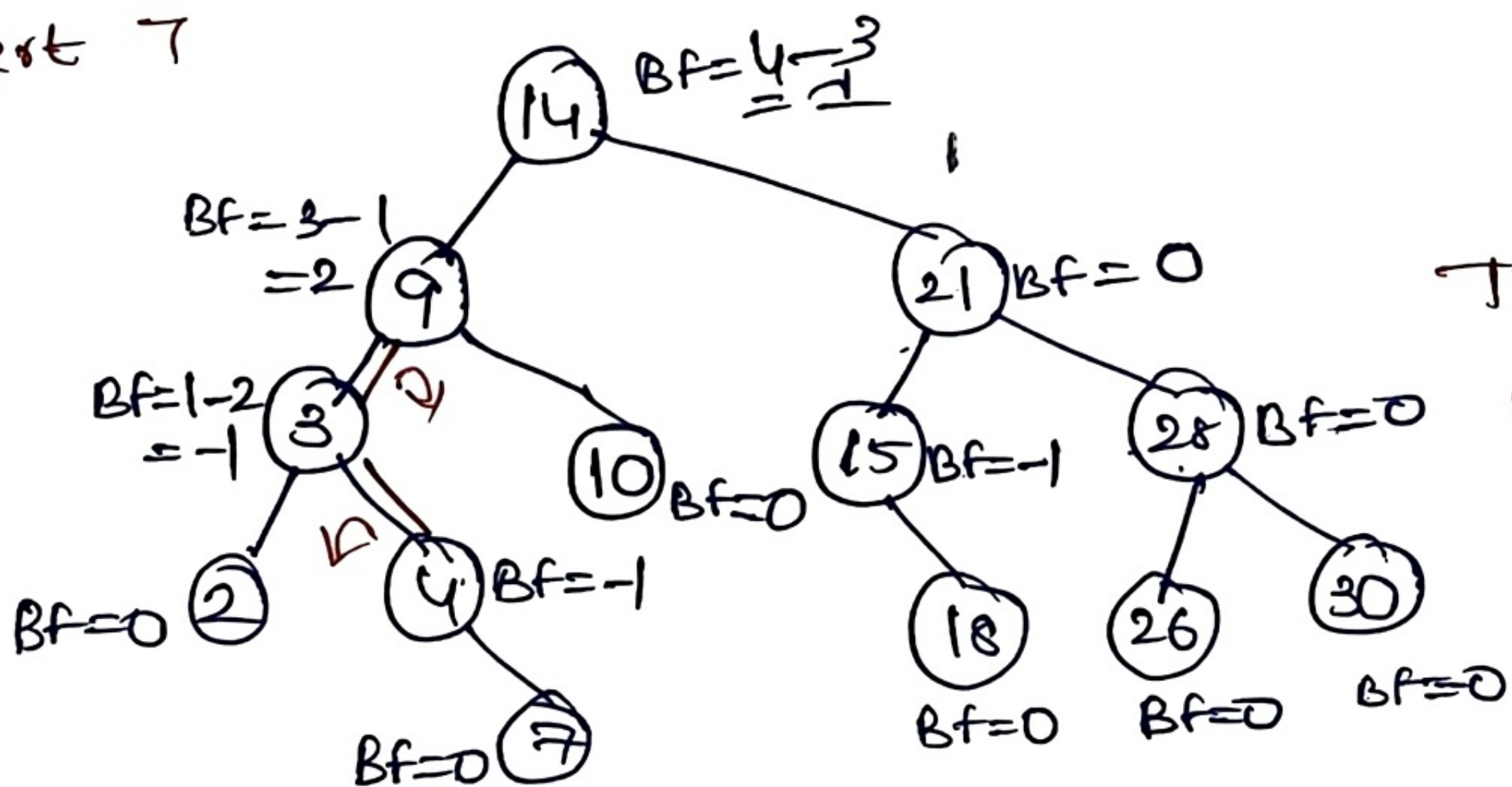
Apply rotations.



Tree is balanced

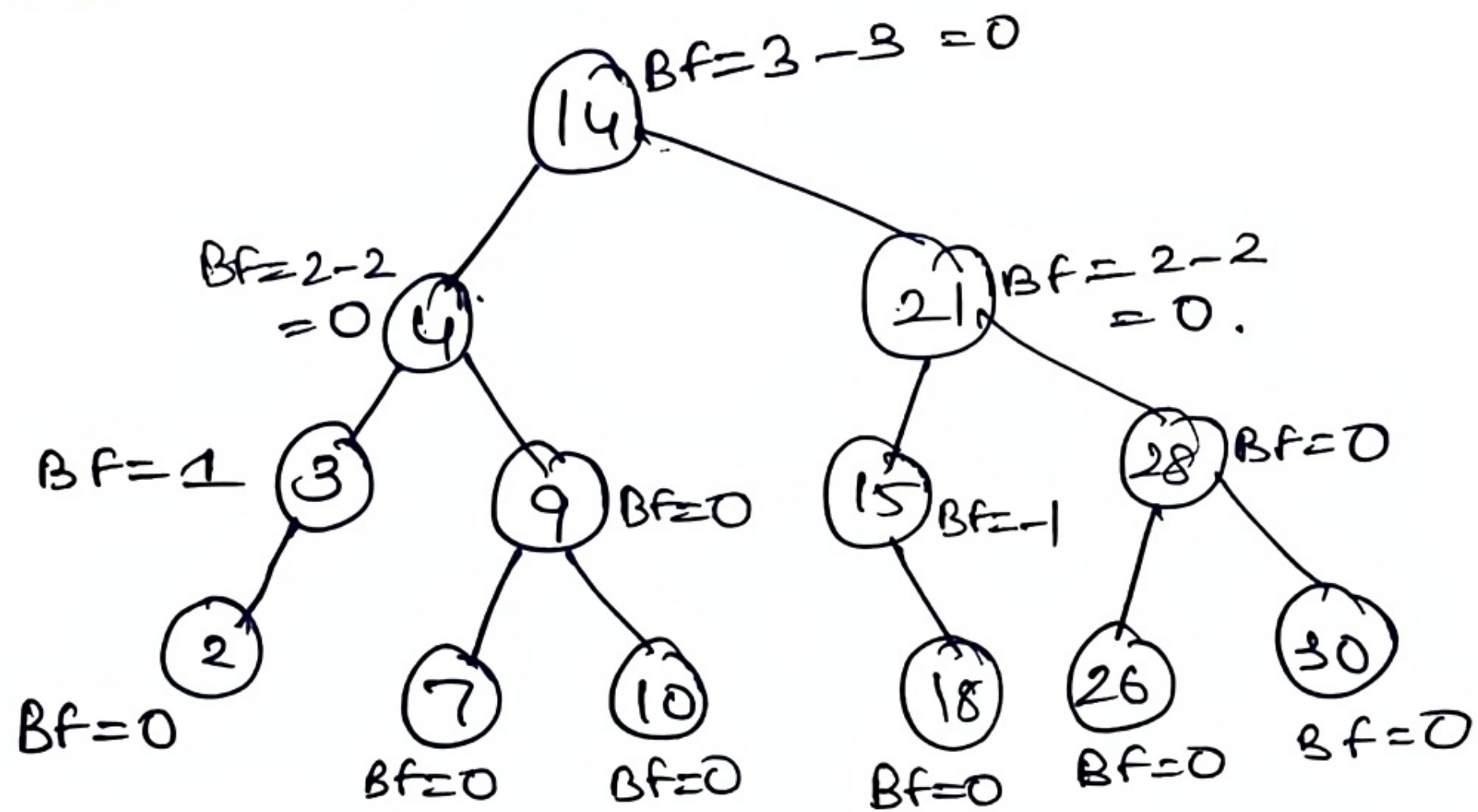
Step 13

Insert 7



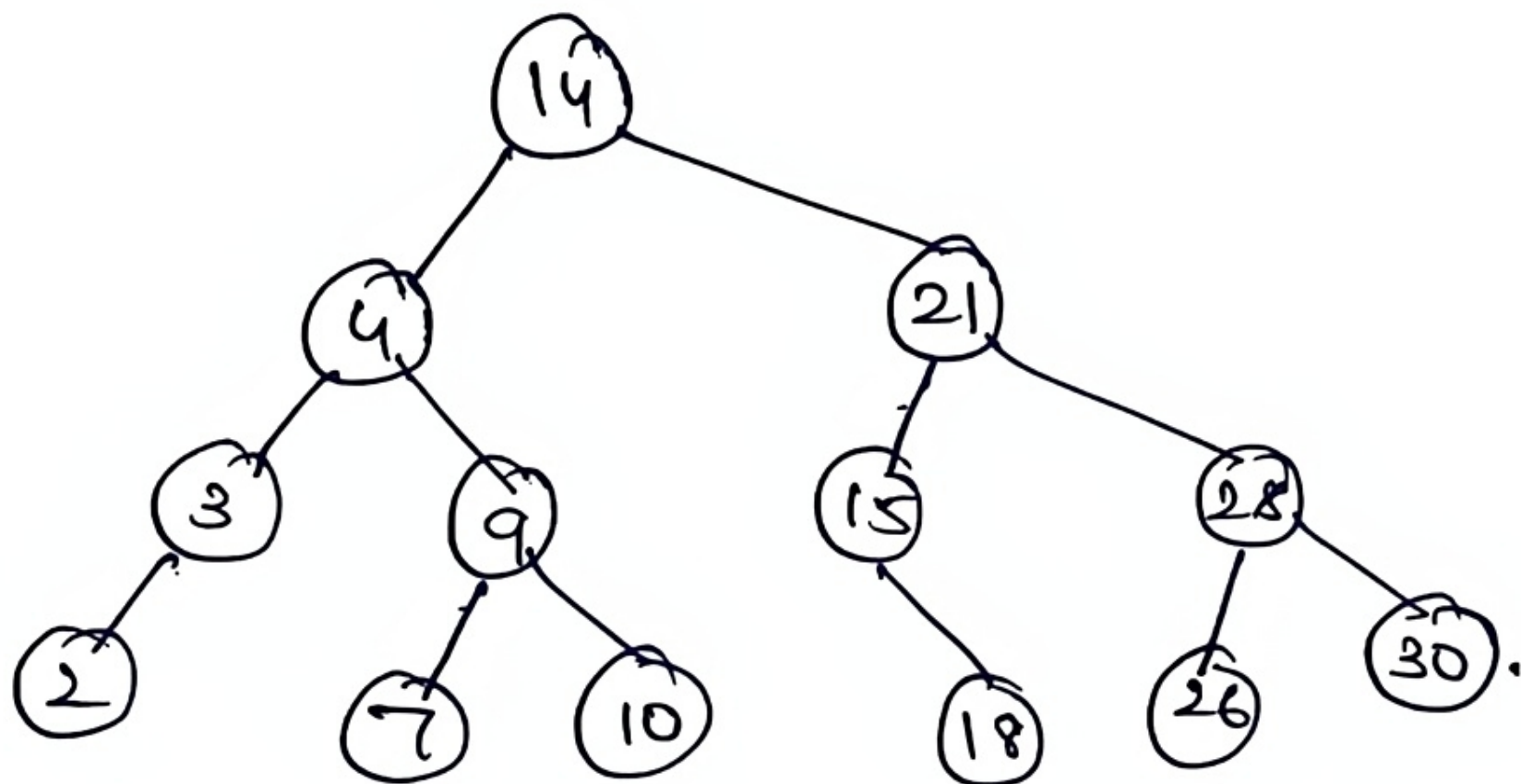
Tree is unbalanced at node (9)

• Apply rotation
 $\Rightarrow$
 LR (RL Rotation)



Tree is balanced.

∴ AVL Tree for above values are



NOTE:

AVL TIME COMPLEXITY:

1. Creation of AVL with n nodes: $O(n \log n)$
2. Insertion: $O(\log n)$
3. Deletion: $O(\log n)$
4. Search: $O(\log n)$
5. Traversal: $O(n)$
6. Rotation: $O(1)$

B-TREE

①

Definition:

- * A B Tree is a self balancing multiway search tree (m-way tree) in which each node can contain multiple keys and have multiple children.
- * It maintains Sorted data.
- * All leaf nodes must be at same level.
- * It is widely used in databases and file systems because it minimizes disk accesses.

Note:

A B-Tree stores multiple keys in each node, making the tree shorter and reducing the number of accesses needed to find data.

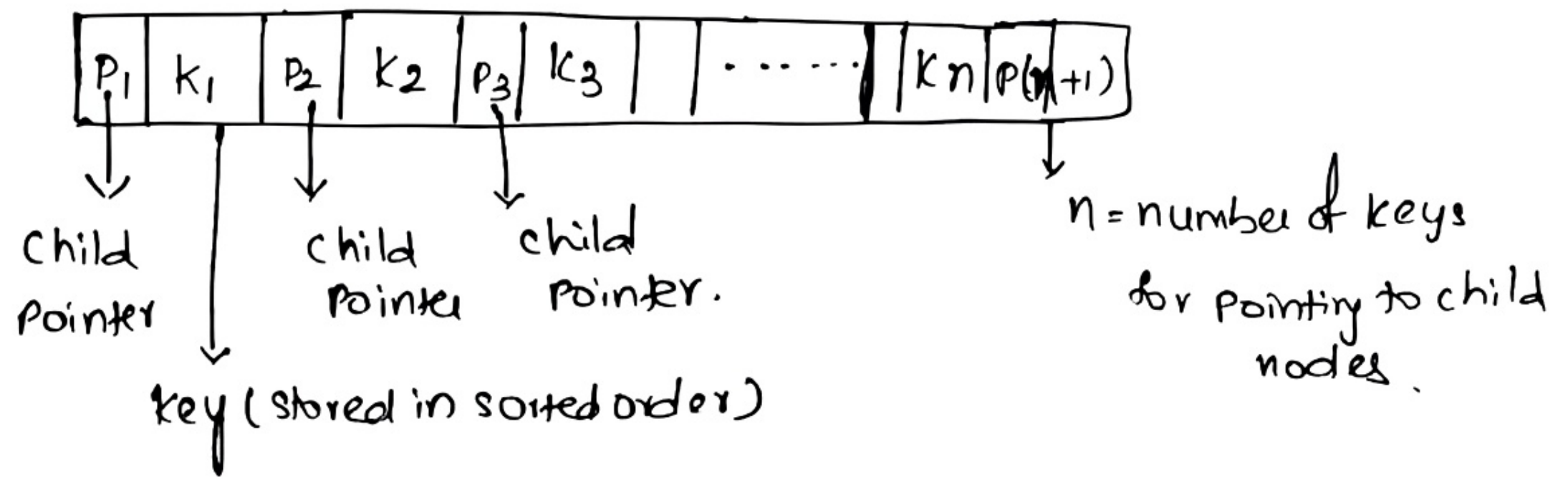
Node Structure of a B-Tree:

- * A B tree node stores multiple keys and child pointers.
- * The number of keys and children depends on the order of the B-Tree

General Node Structure:

for B-Tree of order m :

- * Maximum children = m
- * Maximum keys = $m-1$
- * No. of child pointers = No. of keys + 1



P = pointers to child nodes

K = keys (array to store sorted keys)

n = number of keys.

PROPERTIES OF B-TREE

1. A B-Tree is a self-balancing multiway search tree.

2. Maximum children:

A B-Tree of order m can have at most m children

3. Minimum children:

Every internal node (except the root) must have at least $\lceil m/2 \rceil$ children.

4. Maximum keys:

A node can contain at most $(m-1)$ keys

5. Minimum keys:

Every non-root node must have at least $\lceil m/2 \rceil - 1$ keys.

6. Root Node:

* The root must have at least 1 key

* If the root is not a leaf, it must have at least 2 children.

7. Sorted keys:-

②

* keys in every node are sorted in ascending order.

8. Relationship Between keys and children:-

* Node with K children contains $K-1$ keys

9. Balanced Tree:

All leaf nodes are at the same level, so the tree is always balanced.

10. Efficient Operations:

Search, insertion, and deletion operations take

$O(\log n)$ time.

NOTE:

C-STRUCTURE FOR A B-TREE NODE:

#define MAX 3 // maximum no. of keys (Order 4)

Struct BTreeNode

{

int keys[MAX]; // Array of keys

Struct BTreeNode *child[MAX+1]; // child pointers

int n; // No. of keys

int leaf; // 1 $\rightarrow$ if leaf node, 0 otherwise

};

OPERATIONS OF B-TREE:

1. Creation
2. Insertion
3. Deletion
4. Search
5. Traversal.

1. CREATION OF A B-TREE:

Steps;

1. Start with an empty tree.
2. Insert the first key into the root.
3. Continue inserting keys in sorted order.
4. If a node becomes full, split it into two nodes.
5. Move the middle key to the parent.
6. If the root splits, create a new root.

Eg: Let's construct a B-Tree of order 3 ($m = 3$ children & $m - 1$ Maximum keys = $m - 1$)

Insert: 10, 20, 5

$= 3 - 1$
 $= 2$ keys
 per node

Sol: ~~Steps:~~ Given keys: 10, 20, 5

B-Tree order 3

* Maximum children = 3 $\therefore m = 3$

* Maximum keys in a node, $m - 1 = 2$

* If a node get '3' keys, it overflow & must be split.

Step 1: Insert 10

③

The tree is empty, so 10 becomes the root node.

10

Step 2 Insert 20

Compare 20 with 10.

→ 20 is greater than 10

→ Insert it into the same root node in sorted order.

10	20
----	----

∴ Root has two keys 10 & 20.

→ No split is required.

Step 3: Insert 5

Compare 5 with the keys in the root

→ 5 is smaller than 10

→ Insert it in sorted order.

Before sorting:

10	20	5
----	----	---

After sorting

5	10	20
---	----	----

∴ overflow occurs, because a node should have only 2 keys.

Step 4: Split the node.

When overflow occurs:

1. find the middle key.
2. Move the middle key to the parent.
3. Create two child nodes.

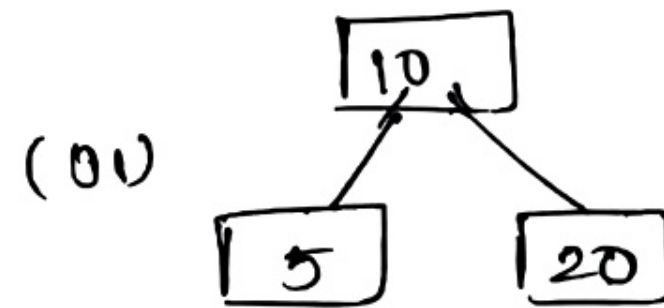
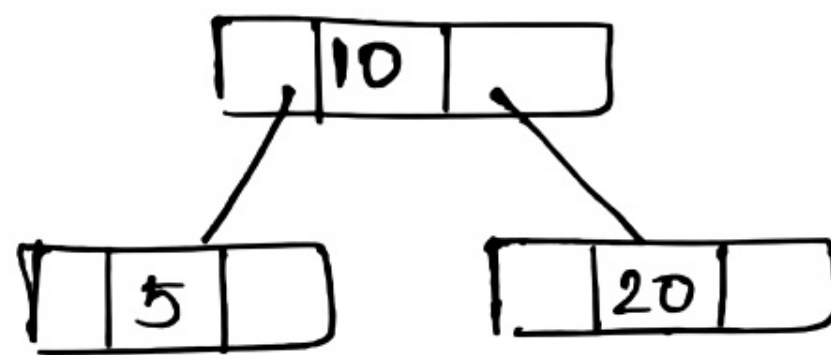
The three keys are:

5	10	20
---	----	----

$$\therefore \text{Apply } m/2 - 3/2 = 1.5 \text{ (ceiling)} \\ = 2$$

Middle key = 10

Move 10 to the parent.



Note:

Go to Google Chrome → B-Trees Visualization ([cs.usfca.edu/ngalles/Visualization/BTree.html](http://cs.usfca.edu/~ngalles/Visualization/BTree.html))

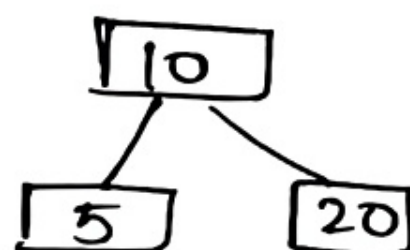
2. INSERTION OPERATION

Algorithm:

1. Search for the correct leaf node.
2. Insert the key in sorted order.
3. If the node overflows:
 - * Split the node.
 - * Move the middle key to the parent.
4. Repeat until no overflow occurs.

Example: Insert 6, 12, 30

Initial tree



④

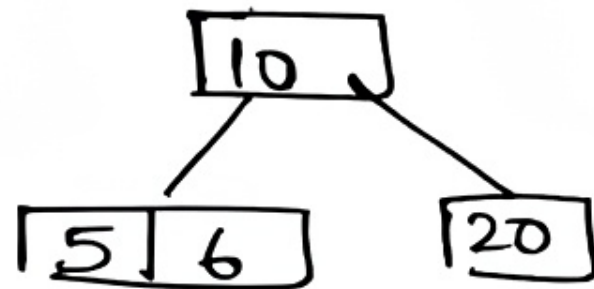
Step 1: Insert 6 for above B-Tree

* Compare 6 with 10

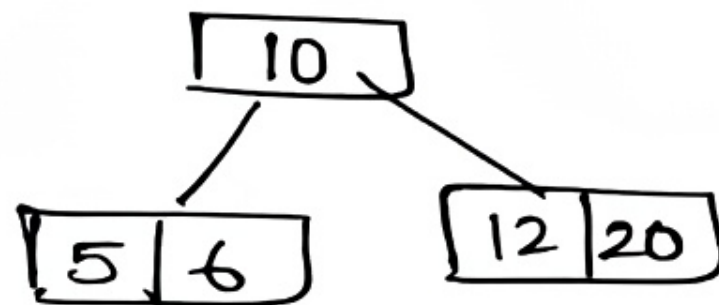
→ Traverse to left

* Compare 6 with 5 (6 > 5)

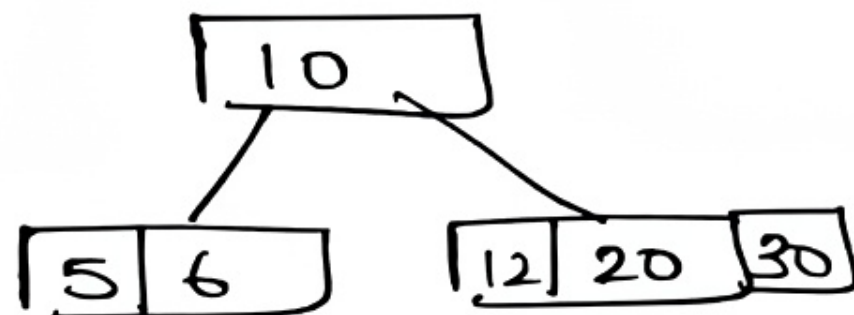
∴ Insert 6 ~~after~~ '5'



Step 2: Insert 12



Step 3: Insert 30



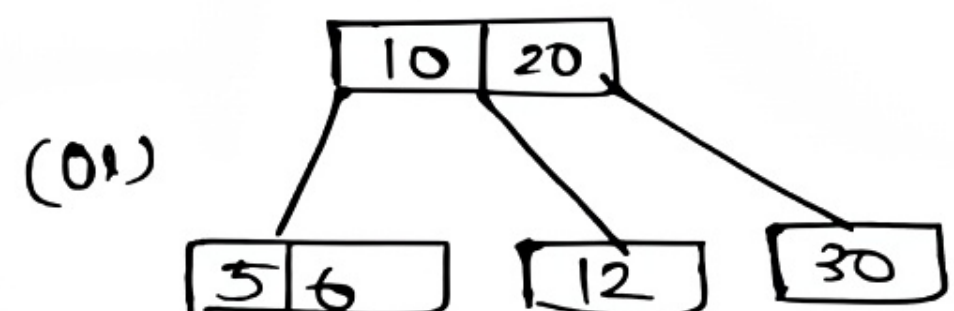
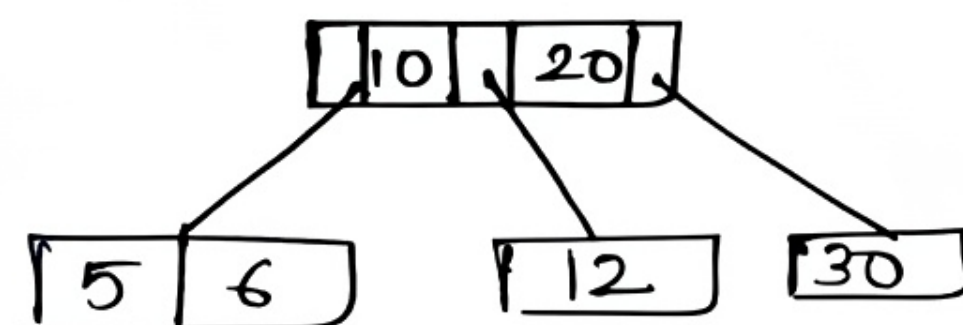
∴ Overflow occurs

Split the node for the keys 12, 20, 30 are:

$$m/2 = 3/2 = 1.5 = 2$$

Middle key for '2' is : 20

Move 20 to the parent

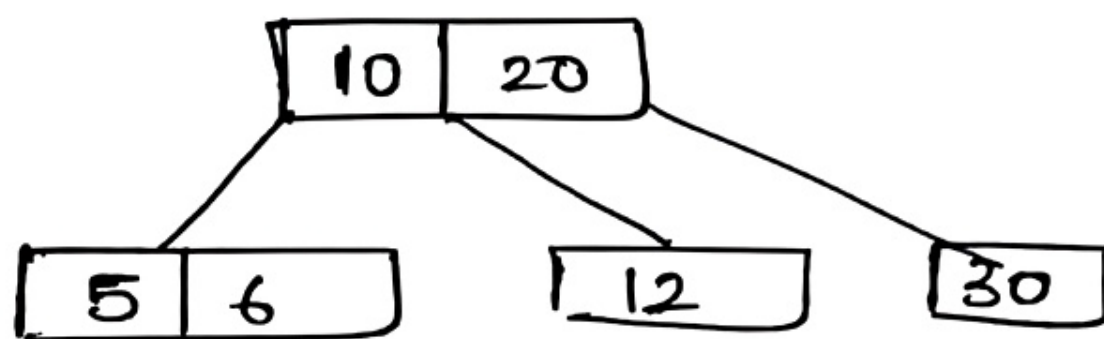


3. DELETE OPERATION:

Steps:

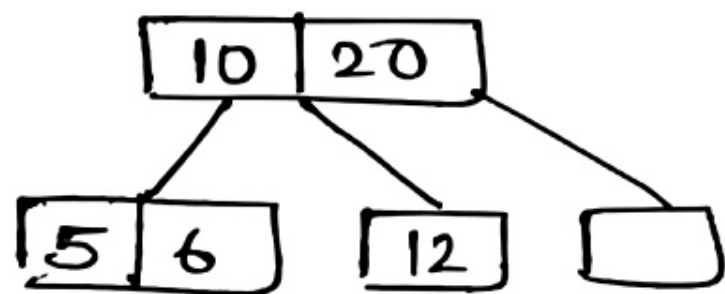
1. find the key to delete.
2. If the key is in a leaf node, remove it
3. If the key is in an internal node:
 - * Replace it with its Predecessor (or) Successor.
4. If a node has ~~too~~^{too} few keys:
 - * Borrow a key from a sibling or
 - * Merge with a sibling.
5. Continue until the tree satisfies all B-Tree properties.

Eg: Initial tree



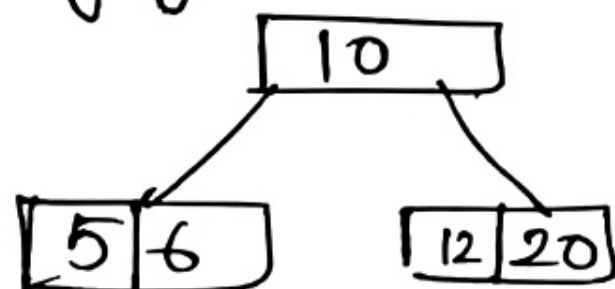
Delete key: 30

Step 1: Key '30' is in leaf node $\Rightarrow$ Remove it



The right child has too few keys, so merge with its sibling

After merging

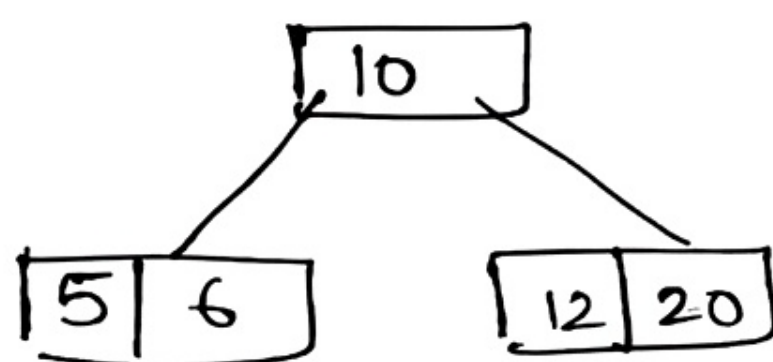


4. Search:

5

1. Used to find a key in the B-Tree.
2. Compare the key with the keys in the current node.
3. move to the appropriate child until the key is found or the search fails.

eg: Initial tree



Search key: 12

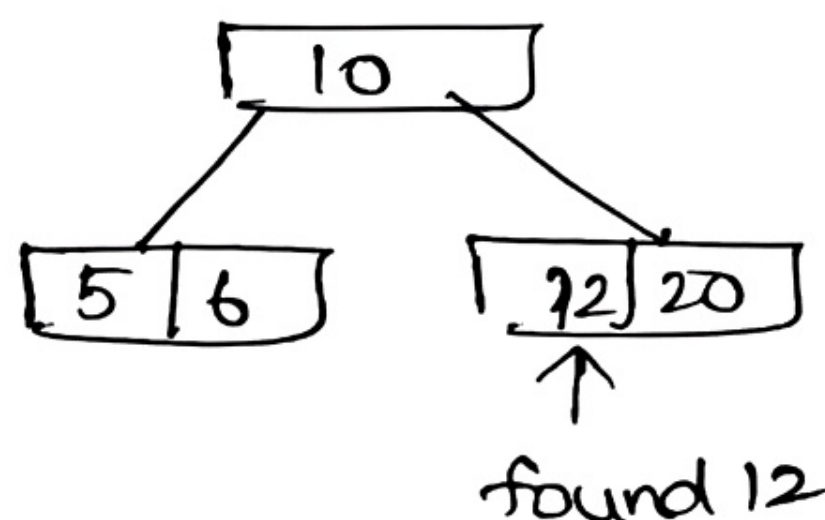
Step 1: Start from root node '10'.

Compare 12 with 10 $12 > 10 \rightarrow$ move to right child.

Step 2: check right child.

Compare 12 with the keys:

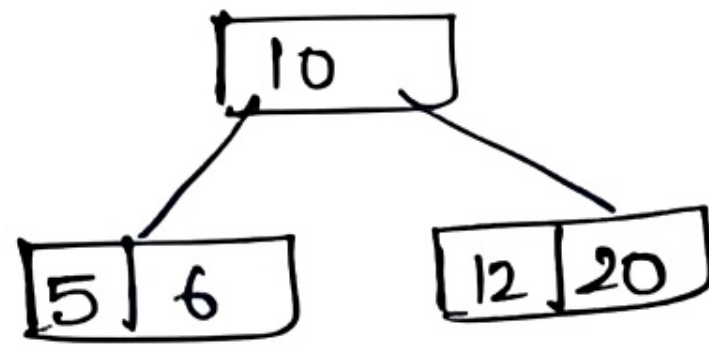
* $12 = 12 \rightarrow$ Key found



5. TRAVERSAL

- * Visit all the keys in sorted order
- * commonly performed using inorder traversal

eg: Initial tree



Inorder Traversal (left $\rightarrow$ Root $\rightarrow$ Right)

$5 \rightarrow 6 \rightarrow 10 \rightarrow 12 \rightarrow 20$

TIME COMPLEXITY FOR B-TREE:

1. CREATION OF B-TREE : $O(n \log n)$

$\rightarrow$ Single insertion : $O(\log n)$

$\rightarrow$ Creating B-Tree with n keys: $n \times O(\log n) \Rightarrow O(n \log n)$

2. Insertion: $O(\log n)$

3. Deletion : $O(\log n)$

4. Search : $O(\log n)$

5. Traversal : $O(n)$

APPLICATIONS OF B-TREE:

1. Database Indexing:

* B-Trees are used to create indexes in databases.

* They allow fast searching, insertion and deletion of records.

2. file systems:

* Used in file systems to store and manage

directories and files efficiently.

⑥

* Helps in quick file searching and retrieval.

3. Disk Storage Systems:

* Used in hard disks and SSDs because they reduce the number of disk accesses.

4. Multi level Indexing:-

* Used to create indexes for very large databases where data is stored on secondary storage.

5. Search engines:-

* Helps in storing and retrieving large amounts of indexed data efficiently.

6. Operating Systems:

* Used for managing files, directories and storage structures.

7. Large - Data applications:

* Used in systems that handle huge amounts of data requiring frequent searching and updating.

8. Key-value Storage Systems:

* Used to store and retrieve key-value pairs efficiently.

Solve the B-TREE QUESTIONS

①

1. Construct a B-tree of order 3 by inserting the following keys:
10, 20, 5, 6, 12, 30
2. Construct a B-tree of order 3 for the sequence:
15, 8, 25, 10, 20, 30, 5
3. Insert the following values into a B-tree of order 4:
50, 20, 10, 30, 60, 70, 40, 80.
4. Construct a B-Tree of order 3 & show every split operation:
5, 15, 25, 35, 45, 55, 65.
5. Build a B-tree of Order 5 using the keys:
100, 50, 150, 25, 75, 125, 175, 200
6. A B-Tree of order 3 is initially empty. Insert the following keys and draw the final B-tree:
45, 12, 78, 34, 23, 56, 90, 11, 67.